FLOWTRIM / COMPLETE REPORT

A complete description of the current FlowTrim application, its purpose, modules, technology, architecture, business rules, implementation standards, local operation and verified delivery status.

Status: MVP complete with post-MVP visual, localization and interaction hardening delivered through EP-01, EP-02 and EP-03.

Prepared for Ander
October 2026

1. Executive summary

What exists today and what has been delivered.

FlowTrim is a full-stack personal-finance web application focused on recurring digital subscriptions. It helps a person centralize subscriptions, understand their real monthly-equivalent cost, anticipate renewals, compare cancellation scenarios and identify services that may be underused.

The product is intentionally deterministic and transparent. It does not connect to banks, does not confirm real transactions, does not verify user activity and does not cancel third-party services. The user owns the data and FlowTrim explains how each number is produced.

The functional MVP is complete. The later experience iterations added a brand identity, landing page, responsive workspace, persisted light/dark themes, English/Spanish localization, landing motion, fixed navigation, isolated table scrolling, reusable CRUD modals, icon actions and dirty-form protection.

Area	Current status
Backend	ASP.NET Core .NET 10 API, PostgreSQL persistence, 90 passing backend tests
Frontend	Angular 21 standalone SPA, 16 passing frontend tests, production build passing
Database	PostgreSQL 17 via Docker Compose, EF Core migrations applied explicitly
UX	Landing, workspace shell, themes, localization, modal CRUD and responsive layout delivered
Delivery	Frontend main and tracker main include EP-03; backend main includes HU-06

2. What is FlowTrim?

Product introduction and user value.

FlowTrim addresses a familiar financial problem: recurring services are easy to start and easy to forget. A monthly charge may feel small in isolation, but a portfolio of subscriptions creates a predictable burn rate that is difficult to see when billing cycles, currencies and renewal dates are distributed across different providers.

FlowTrim turns that invisible recurring spend into an explicit control surface. A user can register a subscription, see the normalized cost, understand what is due soon, record declared usage and simulate what could happen if selected services were trimmed.

Product objective

- Give a person one place to see every recurring subscription, its amount and next billing date.
- Show the real monthly-equivalent cost and annual projection.
- Prevent surprise renewals with visual countdowns and alert windows.
- Reveal potentially underused services through declared cost-per-use information.
- Compare cancellation scenarios without persisting a cancellation.
- Demonstrate a complete, testable full-stack portfolio application.

Product principles

- Transparency: every displayed number should have a visible formula or source.
- Actionability: compare scenarios instead of only listing expenses.
- Honesty: distinguish expected charges from confirmed transactions and declared usage from verified usage.
- Technical simplicity: keep one deployable backend and avoid speculative services.

- Demonstrable quality: tests, validation, error handling and reproducible local setup are part of the product.

3. Main capabilities

What a user can do with the current product.

Capability	Description
Subscription management	Create, list, search, edit and delete subscriptions.
Financial summary	See monthly equivalent, annual projection and active services per currency.
Upcoming charges	Review overdue, today, 3-day, 7-day and later renewal windows.
Declared usage	Record dated usage counts and optional notes for a subscription.
Cost per use	Calculate period cost per declared use without division by zero.
Cancellation simulator	Compare current and simulated spending across 1, 12 and 24 months.
Landing and workspace	Understand the product before entering the responsive financial workspace.
Themes and languages	Use persisted light/dark mode and English/Spanish interface localization.
Modal CRUD	Manage subscriptions through reusable accessible modals with icon actions.

4. Delivered roadmap

Stories and epics completed in the project.

ID	Scope	Status
EP-00	Repository, environment and architecture setup	Complete
HU-01	Register a subscription	Complete
HU-02	View, edit and delete subscriptions	Complete
HU-03	Financial summary	Complete
HU-04	Upcoming charges and countdown	Complete
HU-05	Declared usage records	Complete
HU-06	Cost per use	Complete
HU-07	Cancellation simulator	Complete
HU-08	Angular to API integration	Complete
HU-09	Validation and error handling	Complete
HU-10	Financial rule tests	Complete
HU-11	Documentation and demo	Complete
EP-01	Visual experience redesign / HU-16 to HU-24	Complete
EP-02	Localization and motion / HU-25 to HU-27	Complete
EP-03	Interaction and responsive quality / HU-28 to HU-32	Complete

Post-MVP stories remain planned rather than implemented: HU-12 authentication and multi-user isolation, HU-13 scheduled external notifications, HU-14 CSV transaction import and HU-15 conditional Python/FastAPI analytics extraction.

5. System architecture

How the solution is organized.

FlowTrim is a modular monolith with a layered .NET backend and a standalone Angular frontend. The backend remains one deployable service. Domain rules are kept independent from infrastructure and the API exposes DTOs rather than EF Core entities.

```
Browser
-> Angular standalone components
-> Typed HTTP services
-> ASP.NET Core Minimal API endpoints
-> Application handlers
-> Infrastructure repositories
-> EF Core / Npgsql
-> PostgreSQL
```

Backend dependency rule

Layer	Responsibility	Allowed dependencies
FlowTrim.Api	HTTP endpoints, DI, CORS, configuration and error pipeline	Application, Infrastructure
FlowTrim.Application	Use cases, DTOs, contracts and orchestration	Domain
FlowTrim.Domain	Entities, value rules and pure financial calculators	None
FlowTrim.Infrastructure	EF Core, PostgreSQL, migrations, repositories and health checks	Application, Domain
FlowTrim.Tests	xUnit tests for domain and application behavior	Tested projects

Frontend structure

- Standalone Angular components with strict TypeScript and strict templates.

- Feature-oriented areas for subscriptions, summary, simulator and landing.

- Core services for HTTP error handling, theme state and localization.

- SCSS and CSS custom properties for Carbon Signal tokens and theme modes.

- Native SVG icon and modal primitives to avoid unnecessary UI dependencies.

- Signals for local reactive state; no competing global state manager.

6. Domain model and business rules

The deterministic financial engine.

Subscription

A Subscription contains an identifier, name, optional category, amount, ISO-style currency code, billing cycle, next billing date, status and audit timestamps. The domain factory validates invariants and does not expose public setters that bypass them.

UsageRecord

A UsageRecord belongs to a subscription and stores a usage date, a non-negative usage count and optional notes. Usage is explicitly declared by the user and is not independently verified.

Financial formulas

Indicator	Rule	Important caveat
Monthly equivalent	Monthly amount; yearly amount divided by 12	Yearly values are rounded to two decimals using AwayFromZero.
Annual projection	Sum of active monthly equivalents multiplied by 12	Calculated per currency; currencies are never converted or combined.
Potential savings	Selected monthly equivalent multiplied by selected horizon	Scenario only; taxes, penalties and future price changes are excluded.
Days until charge	NextBillingDate minus server reference date	Past values are overdue, not future charges.
Cost per use	Period cost divided by total declared uses	No records or zero uses return not calculable, never infinity.

7. Backend API

Routes, contracts and behavior.

Method	Route	Purpose
GET	/api/health	API and database availability
GET	/api/subscriptions?page=&pageSize=	Paged subscription list
GET	/api/subscriptions/{id}	Subscription detail
POST	/api/subscriptions	Create subscription
PUT	/api/subscriptions/{id}	Update subscription
DELETE	/api/subscriptions/{id}	Delete subscription
GET	/api/summary	Financial summary per currency
GET	/api/subscriptions/upcoming	Upcoming charges and alert windows
POST	/api/subscriptions/{id}/usage-records	Register declared usage
GET	/api/subscriptions/{id}/usage-records	List usage records
GET	/api/subscriptions/{id}/cost-per-use	Cost per declared use
POST	/api/simulations	Non-persistent cancellation simulation

HTTP and error standards

- 201 Created is returned for successful subscription and usage creation.

- 204 No Content is returned for successful deletion.

- 400 ValidationProblemDetails is returned for invalid input.

- 404 ProblemDetails is returned for unknown resources.

- Unexpected failures return a generic 500 without stack traces, connection strings or internal details.

- CORS is configurable and defaults to the Angular development origin only.

8. Database and persistence

How PostgreSQL is created and evolved.

Docker Compose runs the official PostgreSQL 17 image. The initial database and database user are created by the image from POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_DB values in the local .env file. The application schema is then created by EF Core migrations, applied explicitly with dotnet ef database update.

```
flowtrim-postgres
PostgreSQL 17
host port: 5433 in the current environment
internal port: 5432
database: flowtrim
named volume: flowtrim_pgdata
```

Tables

Table	Purpose	Important constraints
subscriptions	Recurring services and billing metadata	amount > 0; text checks for billing cycle and status; index on next_billing_date
usage_records	Declared usage history	usage_count >= 0; foreign key to subscriptions; ON DELETE CASCADE
__EFMigrationsHistory	Applied EF Core migrations	Maintained by EF Core

Database identifiers use lowercase snake_case. Money uses numeric(18,2), timestamps use timestamptz, dates use date and enums are stored as readable text with database CHECK constraints.

9. Frontend and visual system

The current FlowTrim experience.

The frontend now has two visual layers. The public landing page introduces the product, explains the problem and shows a sample product preview. The /app workspace provides the operational control surface with a fixed navigation rail, theme toggle, language selector, API status and feature navigation.

Carbon Signal palette

Token	Dark value	Light value	Meaning
Primary	#1FC1CE	#087F89	Actions, focus, brand signal
Success	#00C483	#008F61	Savings and healthy states
Warning	#FFB547	#A86400	Upcoming or caution states
Danger	#FF6B6B	#C63D47	Destructive and error states
Background	#070B0D	#F3F7F6	Application canvas

UX behavior delivered

- Persisted dark/light theme with system preference fallback.
- Persisted English/Spanish language selection with safe English fallback.
- Fixed sidebar on desktop and fixed mobile navigation.
- Scrollable feature rail using native scroll snap.
- Reduced-motion support for non-essential animation.
- Reusable native dialog modal with X, Escape and backdrop close requests.
- Dirty-form protection before discarding create/edit data.

- SVG icon actions with accessible labels and tooltips for CRUD.

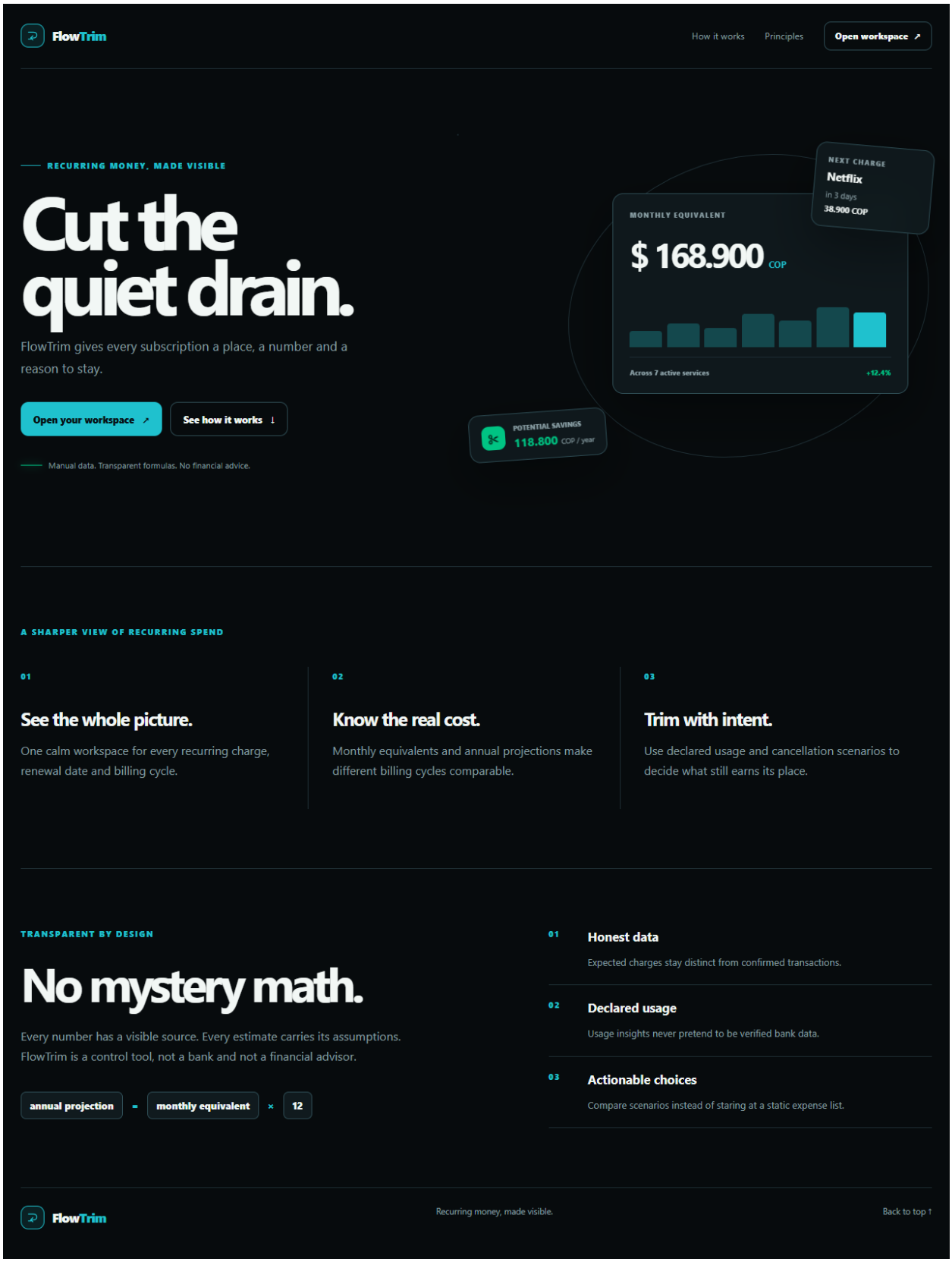


Figure 1. Current FlowTrim landing page with Carbon Signal identity, sample workspace preview and product narrative.

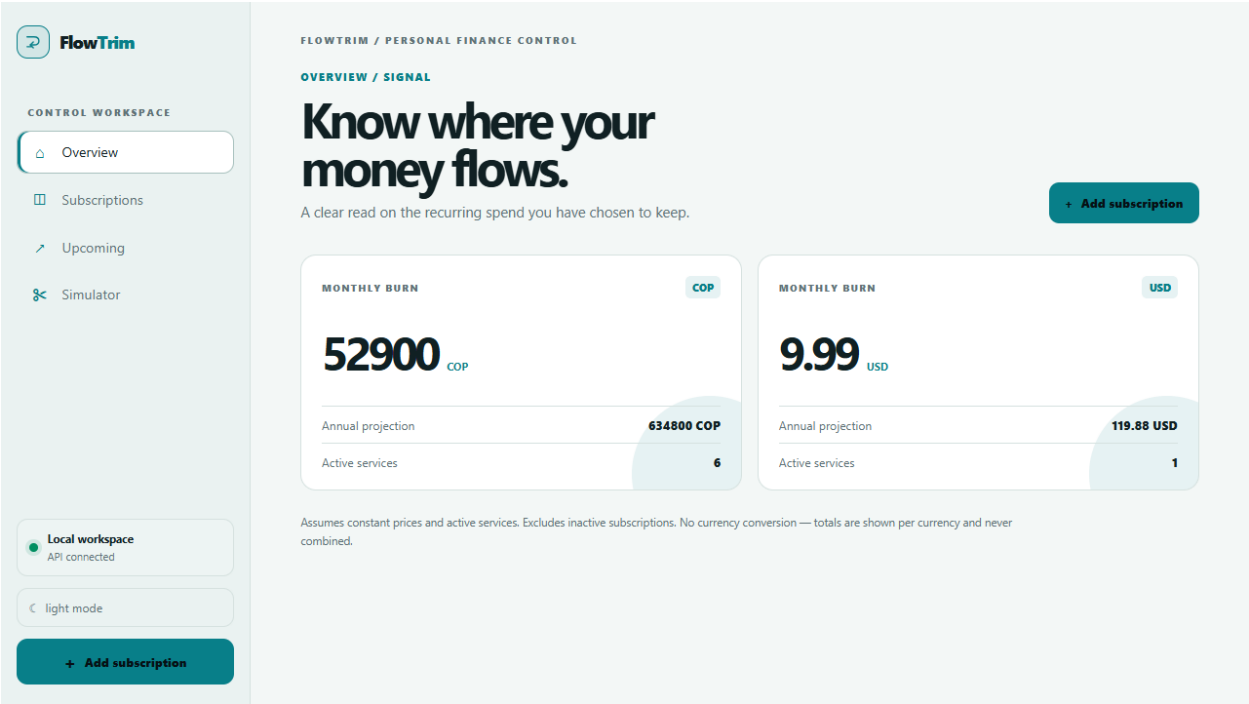


Figure 2. Current workspace overview with fixed navigation, light theme and currency-separated financial cards.

10. Local execution

How to run the complete system on Windows.

Prerequisites

- .NET 10 SDK.

- Node.js compatible with Angular 21; the documented environment uses Node 22.15.0.

- npm 10.

- Docker Desktop and Docker Compose.

Backend and database

```
cd C:\PROJECTS\PROJECTS\FlowTrim\FlowTrim-backend

# Create .env from the template if needed
Copy-Item .env.example .env

# Start PostgreSQL
docker compose up -d
docker compose ps

# Apply EF Core migrations
dotnet tool restore
dotnet ef database update --project src/FlowTrim.Infrastructure --startup-project src/FlowTrim.Api

# Configure User Secrets once
dotnet user-secrets set "ConnectionStrings:FlowTrimDb"
"Host=localhost;Port=5433;Database=flowtrim;Username=flowtrim;Password=YOUR_PASSWORD" --project
src/FlowTrim.Api

# Run the API
dotnet run --project src/FlowTrim.Api --launch-profile http
```

The API runs at http://localhost:5214. The health endpoint should return Healthy for both API and database.

Frontend

```
cd C:\PROJECTS\PROJECTS\FlowTrim\FlowTrim-frontend
npm install
npm start
```

The frontend runs at http://localhost:4200. Use / for the landing page and /app for the workspace.

Useful commands

```
dotnet build FlowTrim.slnx
dotnet test FlowTrim.slnx
npm test -- --watch=false
npm run build
docker compose down
```

11. Database inspection with DBeaver

Connection values for the current local environment.

Field	Value
Database engine	PostgreSQL 17
Host	localhost
Port	5433 in the current environment
Database	flowtrim
Username	Value of POSTGRES_USER in .env
Password	Value of POSTGRES_PASSWORD in .env
Schema	public

In DBeaver open flowtrim -> Schemas -> public -> Tables. The main tables are subscriptions, usage_records and __EFMigrationsHistory.

```
SELECT * FROM public.subscriptions ORDER BY next_billing_date;

SELECT * FROM public.usage_records ORDER BY usage_date DESC;

SELECT s.name, u.usage_date, u.usage_count, u.notes
FROM public.usage_records AS u
JOIN public.subscriptions AS s ON s.id = u.subscription_id
ORDER BY u.usage_date DESC;
```

DBeaver can inspect columns, constraints, indexes and foreign keys. The API does not need to be running for DBeaver to connect; only the PostgreSQL container must be healthy.

12. Standards and engineering principles

Conventions used throughout the project.

- Source identifiers, API routes, database names, filenames and technical identifiers are English.

- Code comments follow the established Spanish project convention; user-facing UI supports English and Spanish.

- Git commits use English Conventional Commits and task branches.

- Specs, new tracker issues and PR descriptions use English for the public portfolio workflow.

- Domain entities protect invariants through factories and controlled mutation.

- Money uses decimal; business formulas are pure and unit-tested.

- I/O is asynchronous and uses CancellationToken in the backend.

- The API never exposes EF Core entities directly.
- Validation exists at API and domain boundaries.
- Errors use RFC 7807 ProblemDetails and do not leak internal details.
- The frontend uses strict TypeScript, strict Angular templates and standalone components.
- User preferences are client-side and do not require backend state.
- Accessibility includes semantic labels, focus states, keyboard behavior and reduced motion.
- Architecture is preserved: no microservices, no competing state manager and no speculative dependencies.

13. Verification and quality evidence

What has been executed successfully.

Check	Result
Backend unit/application tests	90 passed
Frontend unit/service tests after EP-03	16 passed
Frontend production build	Passed without build errors or warnings
Live API health	Healthy API and PostgreSQL connection
Live cost-per-use endpoint	200 calculable, 200 not calculable and 404 unknown subscription verified
Landing browser QA	Rendered with zero console errors and zero failed requests
Workspace browser QA	API-backed overview rendered with zero console errors and zero failed requests
Modal browser QA	Create modal rendered with labels, Cancel first and primary action second
Localization browser QA	Landing changed to Spanish and workspace exposed language control

The project uses an explicit verification loop: implement, build, run unit tests, run the application against PostgreSQL, inspect the UI in a real browser, then document the result.

14. Security, privacy and limitations

What FlowTrim deliberately does not claim.

- No authentication or user isolation exists yet; the application is not ready for public private-data use without HU-12.
- No bank integration, automatic transaction detection or third-party cancellation exists.
- Usage records are declared by the user and are not verified.
- Expected renewal dates are not confirmed bank transactions.
- No currency conversion is performed.
- No emails, SMS or push notifications are sent.
- Public deployment remains deferred because no hosting provider has been selected.
- The backend has a documented NU1903 OpenAPI dependency warning that remains technical debt.
- The frontend keeps Angular 21 and legacy-peer-deps because of the current Node/npm compatibility decision.

15. Pending roadmap

Work that remains intentionally outside the current MVP delivery.

Item	Why it remains pending
HU-12 authentication and multi-user isolation	Required before public use with private data; scope and identity provider are not selected.
HU-13 scheduled external notifications	Requires background execution and an email/push provider.
HU-14 CSV transaction import	Requires a future transaction model distinct from expected subscription renewals.
HU-15 Python/FastAPI analytics	No validated technical need; deterministic C# engine is sufficient today.
Public deployment task #27	Hosting provider and deployment architecture are not selected.
Additional billing cycles and category catalog	Pending product decisions that do not block the current MVP.

16. Delivery history

Key merged delivery points.

Delivery	Result
Backend PR #31	HU-01 register subscription
Backend PR #40 / Frontend PR #22	HU-05 declared usage
Backend PR #41 / Frontend PR #23	HU-06 cost per use
Frontend PR #42 / Tracker PR #38	EP-01 visual experience redesign
Frontend PR #48 / Tracker PR #43	EP-02 localization and motion
Frontend PR #59 / Tracker PR #50	EP-03 interaction and responsive quality
v0.1.0	Initial documented release created after the P0 MVP milestone

Current repository state: backend and frontend main branches are synchronized. The tracker contains the delivered EP-01, EP-02 and EP-03 specifications. A pre-existing untracked specs/_TEMPLATE.md file remains intentionally untouched.

17. Final assessment

Current product maturity.

FlowTrim has moved from a functional MVP into a coherent product experience. Its core strength is the combination of deterministic financial logic, explicit assumptions, reproducible PostgreSQL setup and an increasingly deliberate frontend identity. The next major product threshold is not another visual layer; it is authentication and data isolation before public deployment.

The current system is suitable for local demonstration, portfolio review, technical evaluation and continued feature development. It is not yet positioned as a public multi-user financial service, and the documentation intentionally preserves that distinction.